+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Matplotlib: Personalización avanzada de gráficas

Introducción

Como se indicó en el tema principal de esta semana, Matplotlib es la herramienta de visualización base en Python, gran parte de las herramientas más usadas son construidas sobre Matplotlib debido a su gran versatilidad y personalización. Por tal motivo, es crucial conocer en profundidad los componentes que Matplotlib tiene para la creación de figuras y cómo podemos modificar elementos para crear gráficas según nuestras preferencias y necesidades. En el tema principal usamos el modo indirecto de Matplotlib o *interfaz state-based* para la creación de gráficas. Sin embargo, esto nos limita en la personalización y creación de gráficas. Por lo tanto, en esta guía de profundización usaremos el modo directo o *interfaz object-oriented* el cual nos permite y abre un amplio número de posibilidades y opciones de personalización.

Contenido

A continuación, se abordarán los principales elementos para la creación y personalización de gráficos en Matplotlib.

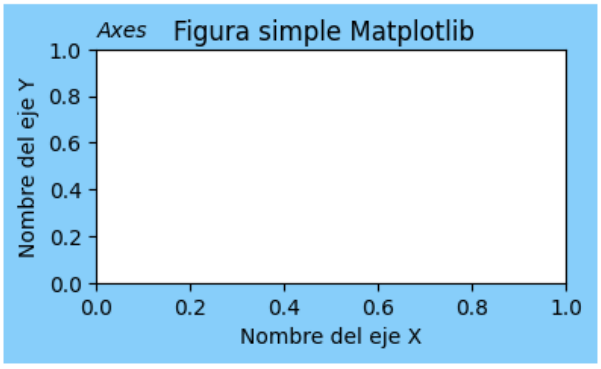
Figuras y lienzos (subplots): Matplotlib gráfica los datos en un elemento llamado Figura (*Figure*) y cada figura contiene uno o más ejes (*Axes*), un área donde los puntos pueden ser especificados en coordenadas *x-y*, *x-y-z* o coordenadas *theta-r* en gráficos polares y además donde coexisten todos los elementos visuales (*Artist*). Iniciamos por la forma más básica para la creación de una figura y algunas opciones de personalización para más adelante agregar complejidad.

```
# Forma más básica para la creación de una figura
# El facecolor color que rodea la figura
fig = plt.figure(figsize=(4, 2), # Tamaño
                 facecolor='lightskyblue')
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
# Agregamos un subtítulo para la figura
fig.suptitle('Figura simple Matplotlib')
# Agregamos ejes
ax = fig.add_subplot()
# Agregar un título al lienzo - Posición izquierda
# Tipo de fuente oblicua - Tamaño de fuente media
ax.set_title('Axes',loc='left',
             fontstyle='oblique', fontsize='medium');
# Añadimos nombres a los ejes
ax.set_xlabel('Nombre del eje X');
ax.set_ylabel('Nombre del eje Y');
```

Figura 1: Figura simple en Matplotlib con un único lienzo



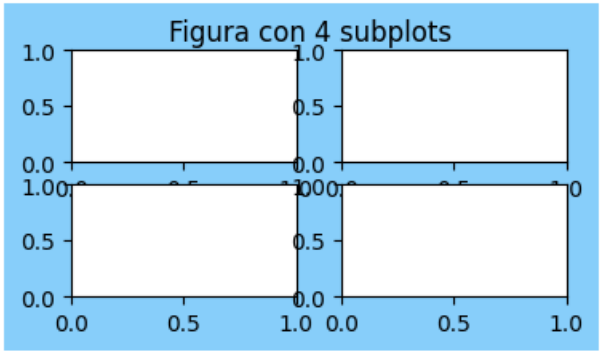
Fuente: Elaboración propia.

La figura resultante muestra como es la forma de crear una figura (especificando el tamaño (ancho, alto)) y añadir algunas opciones de personalización como asignar nombres a los ejes *x* y *y*, tipos de letra y tamaño de la fuente (*xx-small, x-small, small, medium, large, x-large, xx-large, larger, smaller*). La figura anterior solo tiene un eje y fue añadido después de crear la figura. Hay otra alternativa para añadir múltiples ejes desde el principio usando pyplot.subplots.

```
# Añadimos 4 subplots distribuidos en dos columnas y dos filas
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(4,2),
                         facecolor='lightskyblue')
# Agregamos un subtítulo para la figura
fig.suptitle('Figura con 4 subplots');
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 2: Figura en Matplotlib con 4 lienzos



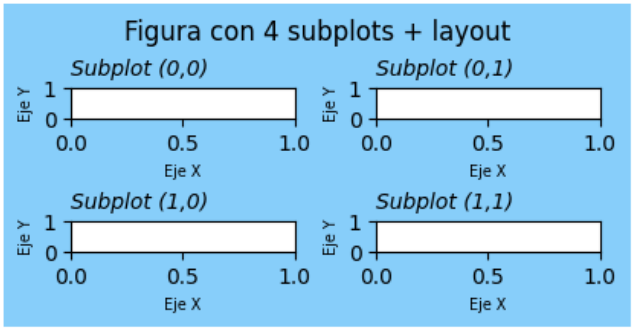
Fuente: Elaboración propia.

El resultado que tenemos es una figura con 4 lienzos para pintar, pyplot.subplots recibe un parámetro que indica cuantos lienzos deseo crear distribuidos en filas y columnas. En nuestro ejemplo, indicamos 2 filas y columnas resultando en 4 lienzos. Además, estos lienzos pueden ser manejados independientemente a través de la variable *axes*. Esta variable es un arreglo en forma de matriz de nfilas x ncolumnas, así que acceder a cada lienzo es cuestión de especificar qué columna y fila deseo usar, miremos un ejemplo:

```
# Importancia de usar el parámetro layout
# Añadimos 4 subplots distribuidos en dos columnas y dos filas
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(4,2),
                         facecolor='lightskyblue',
                         layout='constrained')
# Agregamos un subtítulo para la figura
fig.suptitle('Figura con 4 subplots + layout');
# Agregar un título a cada lienzo - Posición izquierda
# Tipo de fuente oblicua - Tamaño de fuente media
for row in range(2):
  for col in range(2):
    axes[row][col].set_title(f'Subplot ({row},{col})',
                 loc='left', fontstyle='oblique',
fontsize='medium');
    # Añadimos nombres a los ejes
    axes[row][col].set_xlabel('Eje X', fontsize='x-small');
    axes[row][col].set_ylabel('Eje Y', fontsize='x-small');
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 3: Figura en Matplotlib con 4 lienzos sin sobrelapamiento



Fuente: Elaboración propia.

Al final de este ejemplo (en el bucle for), podemos ver cómo se accede a cada lienzo de manera individual, para añadirle el título a cada lienzo. Además, usamos un parámetro llamado *layout,* el cual evita que los elementos dentro de la figura se sobrelapen entre sí, pero se reduce el tamaño del lienzo, por lo que es necesario aumentar el tamaño de la altura de la figura. Ahora, abordaremos otra alternativa para crear lienzos rápidamente y más personalizados, es decir, sin que la distribución de ejes sea simétrica:

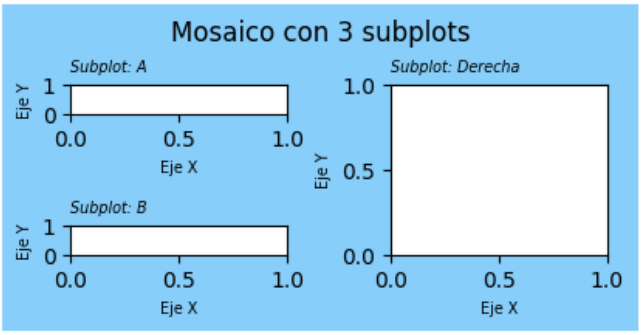
```
# Crear mosaico con 3 lienzos
fig, axes = plt.subplot_mosaic(mosaic=[['A', 'Derecha'],
                                       ['B', 'Derecha']],
                               figsize=(4,2), layout='constrained',
                               facecolor='lightskyblue')
# Agregamos un subtítulo para la figura
fig.suptitle('Mosaico con 3 subplots');
# Los lienzos vienen en un objeto dict de python
for ax_name, ax in axes.items():
  ax.set_title(f'Subplot: {ax_name}',
                loc='left',fontstyle='oblique', fontsize='x-small')
  # Añadimos nombres a los ejes
  ax.set_xlabel('Eje X', fontsize='x-small');
  ax.set_ylabel('Eje Y', fontsize='x-small');
```

Figuras o grillas más complejas pueden ser creadas usando pyplot.subplot_mosaic. En el ejemplo es posible evidenciar cómo se crearon 3

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

lienzos de diferentes tamaños a través de una definición de listas. Algo a notar es que este método no devuelve un arreglo de lienzos sino que devuelve un *dict*.

Figura 4: Mosaico en Matplotlib con 3 lienzos de diferente distribución



Fuente: Elaboración propia.

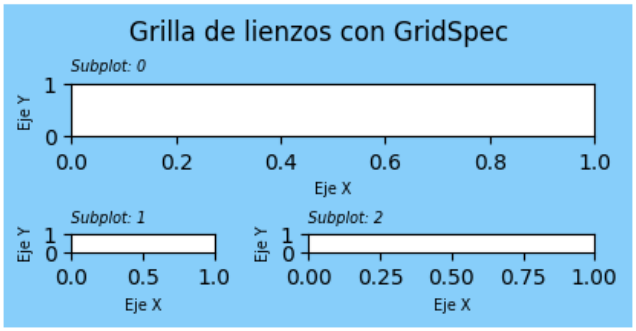
Para entender un poco mejor pyplot.subplot_mosaic podemos revisar la clase que usa por debajo llamada GridSpec. El siguiente ejemplo de más bajo nivel explica cómo crear una grilla de lienzos con más opciones de personalización:

```
# Usar la clase de bajo nivel GridSpec para crear
# una grilla de lienzos similar a lo que hace subplot_mosaic
fig = plt.figure(figsize=(4,2),
                 layout='constrained', facecolor='lightskyblue')
fig.suptitle('Grilla de lienzos con GridSpec')
# Crear el objeto GridSpec
# los ratios de anchura traducen en que la segunda columna es el
doble que la primera mientras que los de altura se traducen en que
# la primera fila es 3 veces mas alta que segunda
gs = fig.add_gridspec(nrows = 2, ncols = 2,
              width_ratios = [1,2], height_ratios = [3,1])
# Añadimos cada lienzo a la grilla para el lienzo ax0 se indica que
debe abarcar toda la primera fila
ax0 = fig.add_subplot(gs[0, :])
ax1 = fig.add_subplot(gs[1, 0])
ax2 = fig.add_subplot(gs[1, 1])
for ax_name, ax in enumerate([ax0, ax1, ax2]):
  ax.set_title(f'Subplot: {ax_name}', loc='left',
                fontstyle='oblique', fontsize='x-small')
  # Añadimos nombres a los ejes
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
ax.set_xlabel('Eje X', fontsize='x-small');
ax.set_ylabel('Eje Y', fontsize='x-small');
```

Figura 5: Grilla con 3 lienzos de distribución diferente usando GridSpec



Fuente: Elaboración propia.

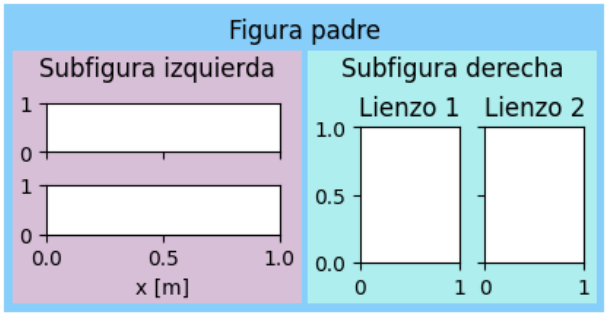
Por otro lado, algunas veces queremos tener un layout anidado a una figura, con dos o más conjuntos de ejes que no compartan la misma grilla. Para esto, es posible usar subfigures que crean figuras virtuales dentro de una figura padre, como se especifica a continuación:

```
# Crear la figura padre o principal
fig = plt.figure(figsize=(4,2), layout='constrained',
                 facecolor='lightskyblue')
fig.suptitle('Figura padre')
# Crear las subfiguras con el n de filas y columnas deseado (es decir
dos subfiguras)
figL, figR = fig.subfigures(nrows = 1, ncols = 2)
# Ajustar el color a la subfigura izquierda
figL.set_facecolor('thistle')
# La subfigura izquierda tiene dos filas
# y una columna y comparte la escala del eje x
axL = figL.subplots(2, 1, sharex=True)
# Colocar el nombre al eje x del segundo lienzo en la subfigura 1
axL[1].set_xlabel('x [m]')
# nombre de la subfigura de la izquierda
figL.suptitle('Subfigura izquierda');
# cambiar el color de la subfigura de la derecha
figR.set_facecolor('paleturquoise')
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
#Añadir dos lienzos a la subfigura de la derecha compartiendo el eje
y
axR = figR.subplots(1, 2, sharey=True)
axR[0].set_title('Lienzo 1'); axR[1].set_title('Lienzo 2')
figR.suptitle('Subfigura derecha');
```

Figura 6: Manejo de subfiguras en Matplolib

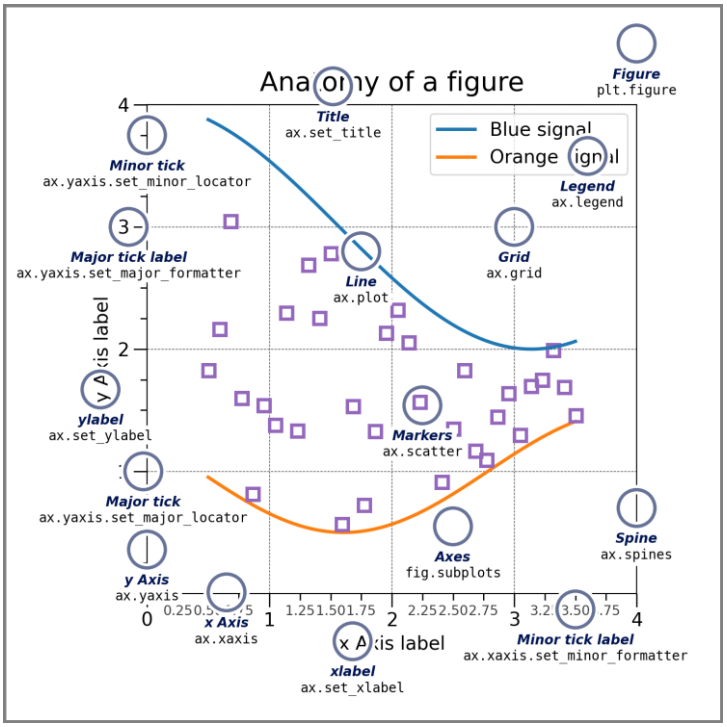


Fuente: Elaboración propia.

De esta manera podemos crear subfiguras. Es importante notar que debemos crear la figura principal y de esta crear las subfiguras para después personalizarlas. En este ejemplo, se usó la propiedad *sharex* y *sharey* que nos permiten compartir la escala de los ejes en los lienzos de cada subfigura. A continuación, la imagen nos permite identificar los elementos de una figura:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 7: Anatomía de una figura Matplotlib



Fuente: Tomado de 'Quick Start guide' por The Matplotlib team, 2023, recuperado de Parts of a figure.

En la siguiente sección abordaremos varias funcionalidades que involucran los elementos presentes en la imagen anterior.

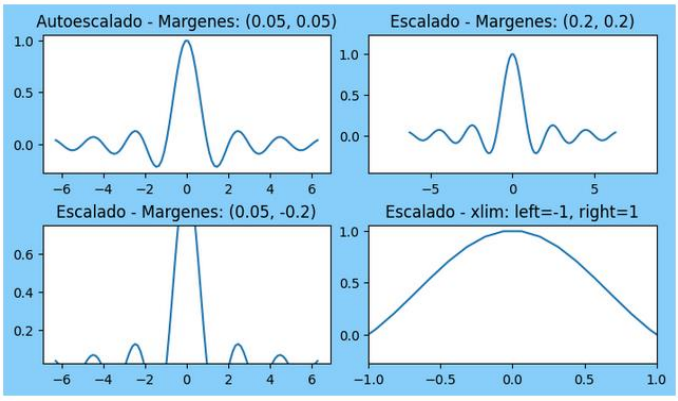
Escalas: Por defecto, siempre que graficamos, Matplotlib fija automáticamente la escala de los ejes basado en los datos, además agrega un margen del 5 % sobre los límites de los datos. Una manera para modificar esto es cambiando el valor de los márgenes del eje (margins) que van desde (-0.5, ∞). Una segunda alternativa es modificando directamente los límites de cada eje. Con esto se desactiva el modo autoescalado. Ejemplo:

```
# Revisemos el escalado automático (5% arriba de los límites)
x = np.linspace(-2 * np.pi, 2*np.pi, 100); y = np.sinc(x)
# Crear la figura
fig, ax = plt.subplots(figsize=(7,4), nrows=2, ncols=2,
                       facecolor='lightskyblue',layout =
'constrained')
ax[0, 0].plot(x,y); # Gráficar
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
ax[0, 0].set_title(f'Autoescalado - Márgenes: {ax[0, 0].margins()}')
# Cambiar los márgenes
ax[0,1].margins(0.2, 0.2)
ax[0,1].plot(x,y);
ax[0,1].set_title(f'Escalado - Márgenes: {ax[0,1].margins()}');
ax[1,0].margins(y=-0.2)
ax[1,0].plot(x,y);
ax[1,0].set_title(f'Escalado - Márgenes: {ax[1,0].margins()}');
# Escalado con limit
ax[1,1].plot(x,y);
ax[1,1].set_xlim(left=-1, right=1);
ax[1,1].set_title(f'Escalado - xlim: left=-1, right=1');
```

Figura 8: Escalado de gráficas en Matplotlib



Fuente: Elaboración propia.

Sobre cada lienzo de la figura es posible modificar la escala de los ejes, podemos especificar sobre el eje *y* o *x* o ambos si se da solo un valor. Entonces tenemos dos opciones una modificando los márgenes u otra modificando directamente el límite del eje con *set_xlim* o *set_ylim*. Otro tema importante a la hora de graficar es el tipo de escala. En Matplotlib por defecto está la escala lineal, pero hay otros tipos como la logarítmica entre otras: *'asinh'*, *'function'*, *'functionlog'*, *'linear'*, *'log'*, *'logit'*, *'symlog*. Ejemplo:

```
# Crear el mosaico
fig, axes = plt.subplot_mosaic([['linear', 'linear-log'],
                                ['log-linear', 'log-log']],
                               layout='constrained',
```

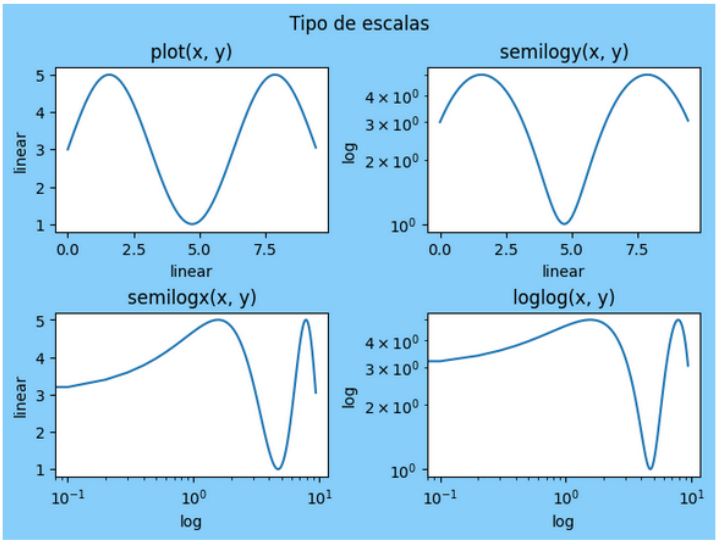
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633


```
                                facecolor='lightskyblue')
fig.suptitle('Tipo de escalas');
# Construir los datos a pintar
x = np.arange(0, 3*np.pi, 0.1)
y = 2 * np.sin(x) + 3
# acceder a los lienzos
ax = axes['linear']
ax.plot(x, y)
ax.set_xlabel('linear')
ax.set_ylabel('linear')
ax.set_title('plot(x, y)');
# cambiar la escala en y
ax = axes['linear-log']
ax.semilogy(x, y)
ax.set_xlabel('linear')
ax.set_ylabel('log')
ax.set_title('semilogy(x, y)');
# cambiar la escala en x
ax = axes['log-linear']
ax.semilogx(x, y)
ax.set_xlabel('log')
ax.set_ylabel('linear')
ax.set_title('semilogx(x, y)');
# cambiar la escala en los dos ejes
ax = axes['log-log']
ax.loglog(x, y)
ax.set_xlabel('log')
ax.set_ylabel('log')
ax.set_title('loglog(x, y)');
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 9: Tipo de escalas en Matplotlib



Fuente: Elaboración propia.

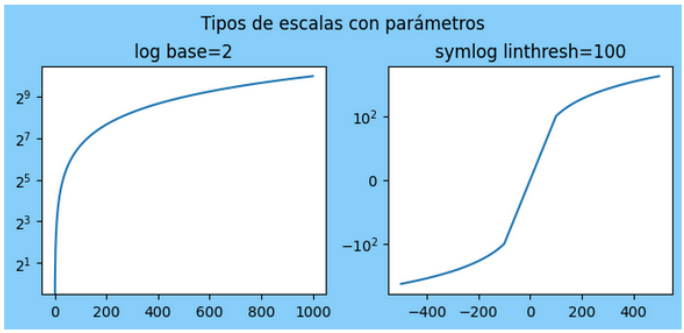
Este es el resultado reemplazando directamente la función plot por el tipo de escala que se desea usar. Sin embargo, otra alternativa, que incluso permite pasar parámetros sería el método *set_xscale* o *set_yscale*. Por ejemplo:

```
# Como pasarle parametros a las escalas,
# por ejemplo, base dos para las logaritmicas
# Crear el mosaico
fig, axes = plt.subplot_mosaic([['log', 'symlog']],
                               layout='constrained', figsize=(6.4,
3),
                               facecolor='lightskyblue')
fig.suptitle('Tipos de escalas con parámetros');
# Crear los datos
x = np.arange(0, 1000)
# Gráficar
ax = axes['log']
ax.plot(x, x)
# cambiar la escala y pasar el parámetro
ax.set_yscale('log', base=2)
ax.set_title('log base=2')
ax = axes['symlog']
ax.plot(x - np.mean(x), x - np.mean(x))
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
# cambiar la escala y pasar el parámetro
ax.set_yscale('symlog', linthresh=100)
ax.set_title('symlog linthresh=100');
```

Figura 10: Tipo de escalas con parámetros en Matplotlib



Fuente: Elaboración propia.

Marcas (Ticks): Los ticks son las marcas en los ejes x y y. Estos dependen de la escala que está siendo usada. Es posible personalizarlos, para tal fin podemos usar métodos de alto nivel como set_xticks o modificando directamente los '*locators*' y "*formatters*". Analizemos un ejemplo usando el método de alto nivel.

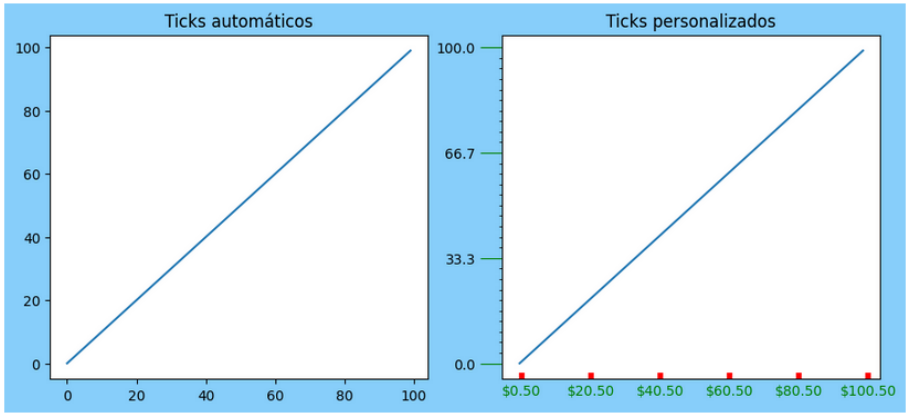
```
# Crear la figura con dos lienzos
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(9,4),

layout='constrained',facecolor='lightskyblue')
x = np.arange(100) # Definir los datos
for nn, ax in enumerate(axes):
    # Gráficar
    ax.plot(x, x)
    # modificar los ticks del segundo lienzo
    if nn == 1:
        ax.set_title('Ticks personalizados')
        # Cambiar los ticks del eje y (marcas principales)
        ax.set_yticks(np.arange(0, 100.1, 100/3))
        # modificar los ticks del eje y (marcas más pequeñas)
        # los valores estarán más cercanos
        ax.set_yticks(np.arange(0, 100.1, 100/30), minor=True)
        # cambiar los ticks del eje x
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
        xticks = np.arange(0.50, 101, 20)
        # Si desea agregar labels personalizados a cada tick
        # es posible a través del parámetro labels
        xlabels = [f'\\${x:1.2f}' for x in xticks]
        ax.set_xticks(xticks, labels=xlabels)
        # Opciones para aplicar estilos a los ticks
        ax.tick_params(axis='x', color='red',
length=4,direction='in', width=4, labelcolor='g', grid_color='b')
        ax.tick_params(axis='y', left=True, color='green',
length=16,
                       grid_color='none')
    else:
        ax.set_title('Ticks automáticos')
```

Figura 11: Personalización de ticks en Matplotlib



Fuente: Elaboración propia.

De esta manera podemos modificar varios atributos de los marcadores no solo de escala si no también visuales según nuestras necesidades. Puedes encontrar más opciones de personalización en: tick_params. En la mayoría de casos lo anterior es suficiente para modificar los ticks. Sin embargo, cuando hay necesidades más específicas podemos hacer uso de funciones de más bajo nivel usando "*Locator*" y "*Formatter*" como en el siguiente ejemplo:

```
# Usando locator y formatters
# La función setup aplica personalización a todos los ejemplos
def setup(ax, title):
```

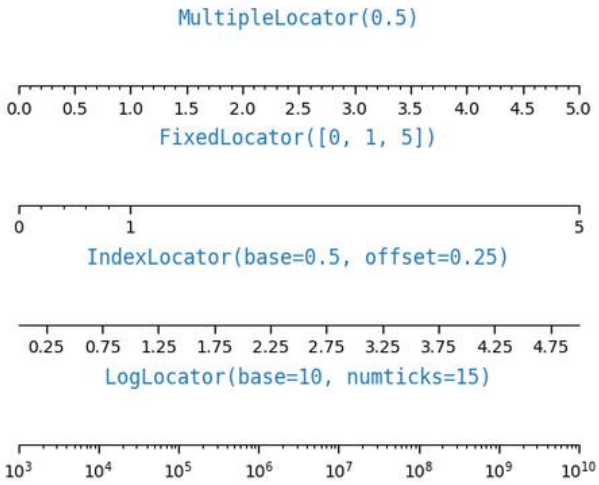
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
    # Spine hace referencia a las líneas del cuadrado de la figura
    # En este ejercicio solo se deja visible la línea de abajo
    ax.spines[['left', 'right', 'top']].set_visible(False)
    # eliminar las marcas del eje y
    ax.yaxis.set_major_locator(ticker.NullLocator())
    # Esta opción nos permite posicionar las marcas en
    # el spin de arriba .. por defecto está en la línea inferior
    ax.xaxis.set_ticks_position('bottom')
    # Para modificar la apariencia de las marcas principales
    # y de las marcas secundarias
    ax.tick_params(which='major', width=1.00, length=5)
    ax.tick_params(which='minor', width=0.75, length=2.5)
    # Fijar los límites para que sea vea mas las diferencias
    ax.set_xlim(0, 5)
    ax.set_ylim(0, 1)
    ax.set_title(title, fontname='Monospace', color='tab:blue')
# Crear la figura
fig, axs = plt.subplots(figsize=(5,4), nrows=4, ncols=1,
                        layout='constrained')
# Multiple Locator
# Coloca las marcas separadas por la distancia
# especificada tanto para los major ticks como para minor ticks
setup(axs[0], title="MultipleLocator(0.5)")
axs[0].xaxis.set_major_locator(ticker.MultipleLocator(0.5))
axs[0].xaxis.set_minor_locator(ticker.MultipleLocator(0.1))
# Fixed Locator
# Esta alternativa permite indicar cuales marcas quiero graficar
setup(axs[1], title="FixedLocator([0, 1, 5])")
axs[1].xaxis.set_major_locator(ticker.FixedLocator([0, 1, 5]))
axs[1].xaxis.set_minor_locator(ticker.FixedLocator(np.linspace(0.2,
0.8, 4)))
# Index Locator - Especificar separado según necesidades
setup(axs[2], title="IndexLocator(base=0.5, offset=0.25)")
axs[2].plot(range(0, 5), [0]*5, color='white')
axs[2].xaxis.set_major_locator(ticker.IndexLocator(base=0.5,
offset=0.25))
# Log Locator
# Graficar en escala logarítmica la base deseada y # de marcas
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
setup(axs[3], title="LogLocator(base=10, numticks=15)")
axs[3].set_xlim(10**3, 10**10)
axs[3].set_xscale('log')
axs[3].xaxis.set_major_locator(ticker.LogLocator(base=10,
numticks=15)); plt.show();
```

Figura 12: Tipos de personalización de Ticks en Matplotlib



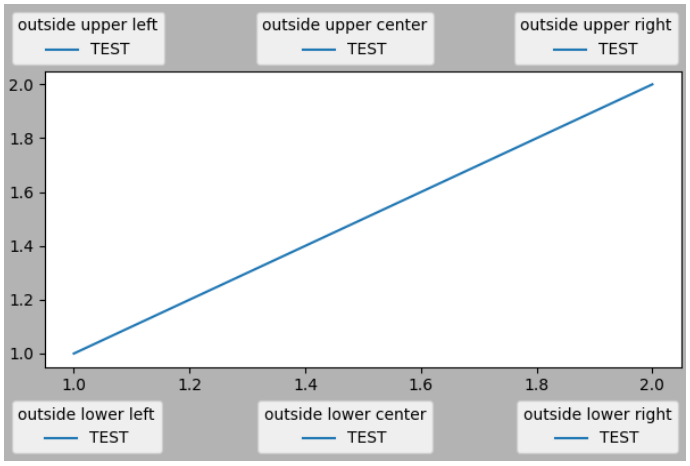
Fuente: Elaboración propia.

Puedes encontrar todas las opciones disponibles: Tick locating and formatting.

Leyendas y Estilos de líneas: Continuamos con aspectos importantes de visualización como lo son las leyendas, estilos de línea y marcadores. Estos elementos son un gran componente visual a la hora de graficar. Con respecto a las leyendas, es importante entender en donde posicionarlas. La siguiente imagen muestra las posibles opciones de posicionar las leyendas en una figura:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 13: Ubicaciones posibles de leyenda en una figura Matplotlib



Fuente: Tomado de 'Axes and Subplots' por The Matplotlib development team, 2023, recuperado de Figure legends.

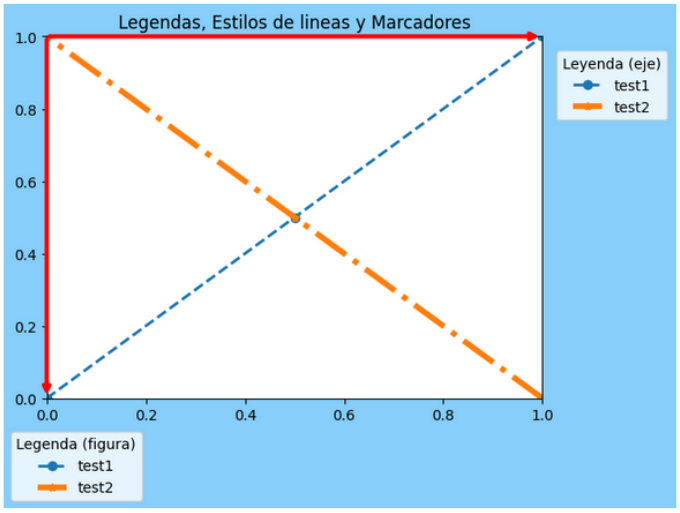
A continuación, se presenta un ejemplo de cómo funcionan estos elementos:

```
# Crear la figura
fig, axes = plt.subplots(facecolor='lightskyblue',
                         layout='constrained')
# Establece los límites del gráfico
axes.set_xlim(0, 1)
axes.set_ylim(0, 1)
axes.set_title('Leyendas, Estilos de líneas y Marcadores')
# Graficar dos líneas de datos con diferentes estilos y marcadores
axes.plot([0, .5, 1], [0, .5, 1],
          label="test1", linestyle='--', linewidth=2, marker='o')
axes.plot([1, 0.5, 0],[0, 0.5, 1], label="test2",
          linestyle='-.', linewidth=4, marker='*')
# colocar el cuadro en una posición específica
# bajo la referencia (x, y, ancho, alto)
# donde x y y pueden ser valores entre 0,1 y
axes.legend(bbox_to_anchor=(1, 1),loc='upper left',
                      borderaxespad=1.00, title='Leyenda (eje)')
# Es posible colocar leyenda a nivel de figura
fig.legend(loc='outside lower left', title='Legenda (figura)');
# Agrega ejes coordenados en la esquina para entender
# el sistema de referencia
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
axes.annotate('',  # No se muestra ningún texto
    (1, 1),  # Punto de inicio del eje x
    (0, 1),  # Punto de finalización del eje x
    arrowprops=dict(arrowstyle='->', color='red', linewidth=3)
)
axes.annotate('',  # No se muestra ningún texto
    (0, 0),  # Punto de inicio del eje y
    (0, 1),  # Punto de finalización del eje y
    arrowprops=dict(arrowstyle='->', color='red', linewidth=3)
)
plt.show()
```

Figura 14: Manejo de leyendas en los lienzos de una figura Matplotlib



Fuente: Elaboración propia.

Lo primero que debemos entender es que es muy fácil cambiar el estilo y los marcadores de línea, solo debemos pasarlo directamente en la función que grafica. Puede encontrar todos los estilos de líneas y marcadores en: Estilos de líneas y Marcadores de líneas. Adicionalmente, como se evidenció es posible cambiar el tamaño y color de las líneas. Por otro lado, está el uso de la leyenda. La manipulación de este elemento puede ser un poco confusa, es por eso que en esta guía se enseña un método sencillo para personalizar la leyenda de cada línea. Hay que saber que para cada elemento graficado en el lienzo es necesario

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

colocarle título a través del parámetro "*label*" y que para que la leyenda se visualice se debe hacer el llamado del método "*.legend*". Aquí viene otro punto importante y es que es posible asignar leyendas tanto a la figura como a los ejes, así como se evidencia en la imagen anterior. Entonces, para la figura ya hay 6 posiciones posibles (ver figura 13). Para los ejes se debe configurar su posición a través del parámetro "*loc*" que es ubicación y la posición (*x,y*) del cuadro de la leyenda con el parámetro "*bbox_to_anchor*". Vale la pena aclarar que este parámetro también recibe valores de altura y anchura pero vamos a ignorar estos valores y nos enfocaremos en la configuración de la ubicación y las coordenadas. Entonces, cuando se selecciona un valor para el parámetro "*loc*" (similar a los valores para la leyenda en figura pero sin la palabra *outside*). Es decir si ingresamos "*lower right*" lo que estamos indicando es que la esquina inferior derecha del cuadro de la leyenda se posicione en la esquina inferior derecha del lienzo (todo esto desde la perspectiva del usuario). Además de entender esto, hay que saber que el eje *x* se mueve de izquierda a derecha de 0 a 1 mientras que el eje *y* se mueve de arriba debajo de 1 a 0 (como indican las flechas rojas). Entonces si estamos ubicados (mentalmente) en la esquina inferior derecha y queremos ubicar nuestra leyenda en la esquina superior derecha habrá que indicar los siguiente valores al parámetro *bbox_to_anchor = (0,1).* Lo anterior indicaría que el vértice inferior derecho del cuadro de la leyenda se ubicaría en el vértice del lienzo con coordenadas x=0, y=1 es decir vértice superior izquierdo del lienzo.

Conclusiones

En esta lectura de profundización se aprendieron diferentes elementos para la configuración y personalización de las gráficas en Matplotlib según las necesidades de nuestros proyectos. En este punto el estudiante ya tiene las herramientas necesarias para seleccionar, configurar y personalizar las gráficas de manera avanzada de acuerdo con el objetivo de la gráfica y del proyecto en general. En detalle, el estudiante puede crear figuras con diferente número de lienzos, tamaños y distribuciones. Además, de la selección y graficación en lienzos de manera individual. También se evidenció cómo personalizar la escala de los ejes del lienzo dependiendo del tipo de dato, lo que asegura una mejor visualización y comprensión. Por otro lado, detalles como el tipo de línea, tamaño de línea, fuente y marcadores permiten la identificación y entendimiento de

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

elementos diferentes en el mismo lienzo para efectos de comparación. Finalmente, se abordó cómo posicionar leyendas tanto en figuras como en ejes usando un método práctico. Una parte importante del proceso de aprendizaje es colocar a prueba y en práctica toda la teoría "aprendida" con el objetivo de afianzar tales conocimientos. Por lo anterior, motivamos al estudiante a que afiance los conocimientos aprendidos en el caso práctico de esta semana. Finalmente, en este enlace hay una guía completa de la opción de personalización en Matplotlib: Personalización en Matplotlib.

Referencia

- The Matplotlib team, (2012-2013). Quick Start Guide. Recuperado el 12 diciembre de 2023 de https://matplotlib.org/stable/users/explain/quick_start.html

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.